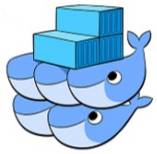
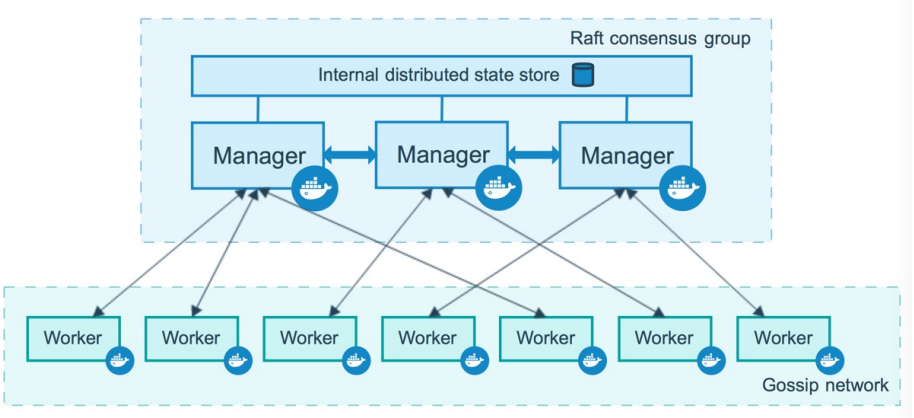
Docker Swarm



- Swarm mode: advanced feature of Docker to natively manage a cluster of Docker engines called a *swarm* https://docs.docker.com/engine/swarm/
- Swarm: multiple Docker engines running in swarm mode
  - Swarm mode helps you orchestrate containers across multiple machines
- Composed of Manager nodes and Worker nodes
  - Manager nodes: control the swarm and handle the orchestration of services
  - Worker nodes: run containers (tasks) as assigned by manager nodes
  - Tasks: containers running in a service
    - Task: smallest unit of work, typically one container
  - Services: how tasks (containers) should run on the swarm; provide an abstraction for deploying and managing tasks

Docker Swarm: architecture

- Node: single instance of Docker engine in a swarm
  - Manager nodes: handles cluster management, including task scheduling
    - Multiple managers to improve fault tolerance
    - Raft as consensus algorithm to ensure consistency
  - Worker nodes execute tasks
    - Workers use a gossip protocol to exchange information about their state



Docker Swarm: features

- Cluster management integrated with Docker
- Decentralized: distributed decision-making
- Declarative service model
- State reconciliation
  - Swarm monitors cluster state and reconciles any differences wrt desired state (e.g., a node crashes)
- Scaling
  - Easily scale services but lacks auto-scaling
- Multi-host networking
  - Use overlay networks to enable communication between services across nodes
- Load balancing
  - Can expose service ports to an external load balancer
- Secure: TLS authentication, encryption, role-based AC
- Easy to use and lightweight

Container-as-a-Service (CaaS)

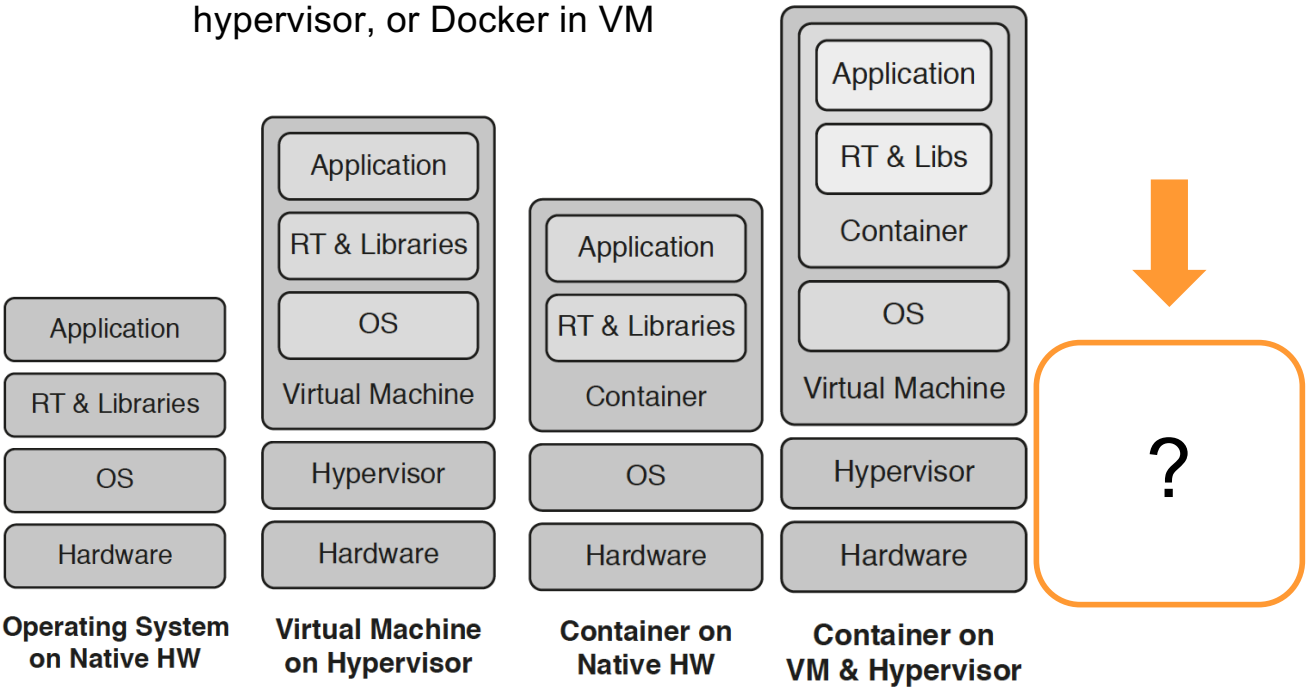
- Cloud-based platform for managing and deploying containerized applications
  - Combines containerization with cloud benefits
- Features
  - Container orchestration and management: container lifecycle, scheduling, load balancing, and fault tolerance
  - Configuration for resource optimization (e.g., auto-scaling)
  - Security and access control
  - Integration with other cloud services (e.g., monitoring)
- Examples
  - Amazon Elastic Container Service https://aws.amazon.com/ecs
  - Azure Container Instances https://azure.microsoft.com/products/container-instances
  - Google Cloud Run https://cloud.google.com/run

Virtualization and IaaS providers

- Which virtualization technology for IaaS providers?
  - ✓ Hypervisor-based virtualization: greater security and isolation, flexibility (different OSs on same PM)
  - ✓ Container-based virtualization: smaller deployment size and higher density, faster startup/shutdown
- Questions
  - Containers on top of bare metal or inside VMs?
    - Performance and density vs. isolation and security
  - Are containers replacing VMs?
    - Not entirely: lightweight vs. isolation and security

New lightweight virtualization approaches

- Deployment approaches examined
  - Plus nested virtualization: hypervisor inside VM on top of an hypervisor, or Docker in VM

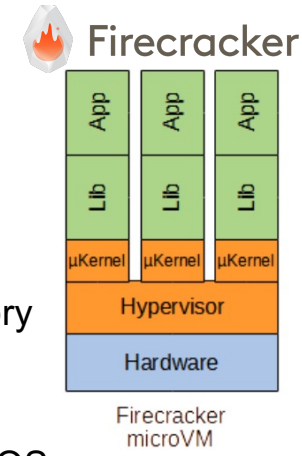


New lightweight approaches to virtualization

- Microservices, serverless computing, cloud-edge continuum demand for
  - Low-overhead (or lightweight) virtualization techniques, even lighter than containers
  - Better security
  - Portability across OSs and architectures (e.g., Arm, Intel)
- Technologies enabling lightweight virtualization: **MicroVM**, **lightweight OSs**, **unikernels** and **WebAssembly**
- MicroVMs, lightweight OSs, and unikernels: reduce OS overhead and attack surface
  - OS overhead: many common OS services (shells, editors, core utils, package managers) are unnecessary
  - Attack surface: images contain only the essential code needed to run the app, reducing potential attack vectors

MicroVM runtimes

- Tiny, specialized VMM that run lightweight VMs (called microVMs)
- Goal: reduce memory footprint and improve security of virtualization layer
- Firecracker: minimalist VMM purpose-built by Amazon for secure, efficient and multi-tenant microVMs
  https://firecracker-microvm.github.io
  - Why? To enable AWS Lambda and AWS Fargate for serverless and containerized workloads
  - Based on KVM but with minimalist design (no unnecessary devices and guest functionality)
  - Open source, written in Rust
  - microVM: <125 ms startup time and <5 MB memory footprint
  - Scales to thousands of multi-tenant microVMs
  - Supported OS guests inside microVM: Linux and OSv

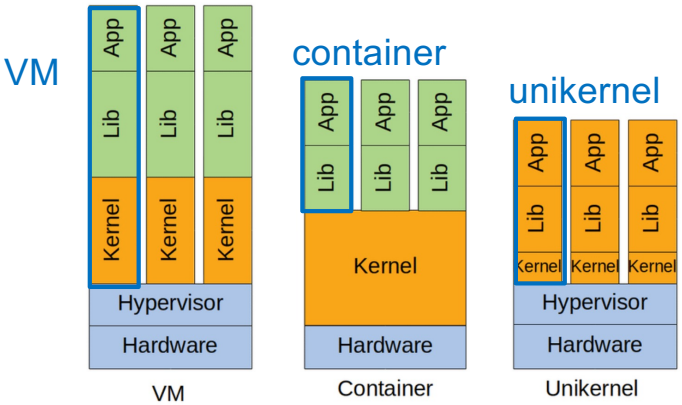


Lightweight operating systems

- Minimal, special-purpose OSs to run containerized apps
- Fedora CoreOS https://fedoraproject.org/it/coreos/
  - Minimal, monolithic, and compact Linux distribution designed for running containers
    - Only components for container deployment, together with built-in tools for service discovery, container management, and configuration sharing
  - Designed for scale and security
  - Fast bootstrap and small memory footprint
  - Can be installed directly on hardware or on hypervisor
  - Includes Docker and podman
- Other products: Ubuntu Core, balenaOS
  - Designed for edge and IoT devices

Unikernels

- Specialized, single-purpose OS designed to run a single application with minimal overhead
  - Single application + OS into a single executable (aka library OS): monolithic process that runs entirely in kernel mode
  - Single address space: app and OS share the same memory space
  - Built by compiling high-level language directly into specialized machine image that runs on hypervisor
  - Goal: isolation benefits of hypervisor without overhead of guest OS



Unikernels: pros and cons

- Pros (specialized → high performance)
  - ✓ Lightweight: less resource-intensive (memory and CPU)
  - ✓ Minimal footprint
  - ✓ Reduced attack surface
  - ✓ Fast execution (no context switching)
  - ✓ Fast boot (measured in ms)
  - ✓ Strong isolation
- Cons
  - ✗ Less flexible than VMs and containers
  - ✗ Limited ecosystem support, including debugging and monitoring tools
  - ✗ Scalability is more challenging (e.g., multi-instance deployments or load balancing)
  - ✗ Unikernel orchestration is not as easy as container orchestration

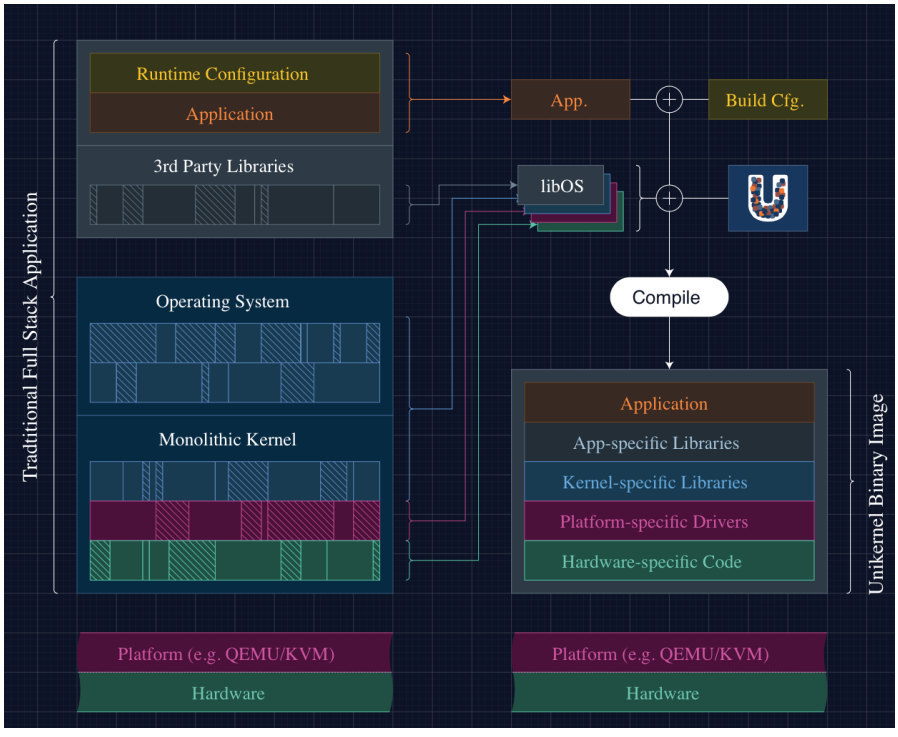
Unikernels: frameworks

- Frameworks
  - MirageOS (Ocaml language) https://mirage.io
  - OSv https://github.com/cloudius-systems/osv
  - Nanos https://nanos.org/
  - Unikraft
- OSv
  - Cloud-native unikernel optimized for high performance and low overhead in virtualized environments
  - Linux ABI compatibility allows running Linux applications with minimal changes
  - Open-source and fast
    - Can boot in ~5 ms on Firecracker using 11 MB of memory

Unikernels: Unikraft

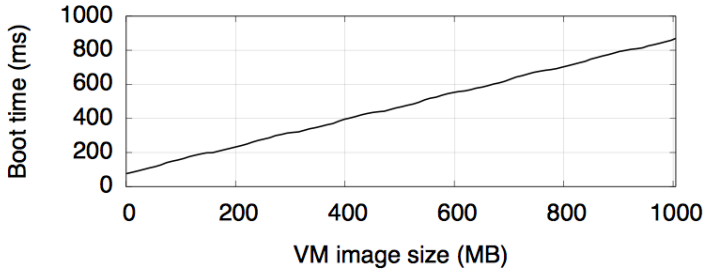
- Fast, secure, and open-source Unikernel Development Kit https://unikraft.org
  - Designed to simplify unikernel creation without requiring deep expertise
  - Build, run and package: similar to Docker
  - Modular: supports a wide range of components (e.g., networking, storage), allowing to create custom unikernels
  - Multi-language support (e.g., C, C++, Rust, Go)
  - Application compatibility
    - Can run complex apps (e.g., Redis, Nginx, Memcached)
    - POSIX compliant: compatibility with a broad range of Unix-like applications
  - Architecture compatibility
    - Works with multiple hypervisors (e.g., Xen, KVM) and supports various CPU architectures, allowing deployment on both virtualized and bare-metal environments
  - Active development

Unikernels: Unikraft

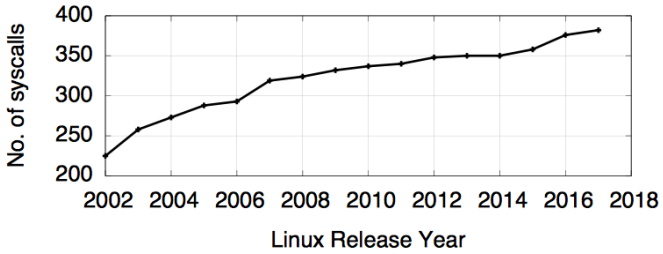


Performance of virtualization approaches

- VM boot times grow linearly with VM size



- Difficulties in securing containers due to growth of Linux syscall API



My VM is lighter (and safer) than your container, SOSP 2017
https://dl.acm.org/doi/pdf/10.1145/3132747.3132763

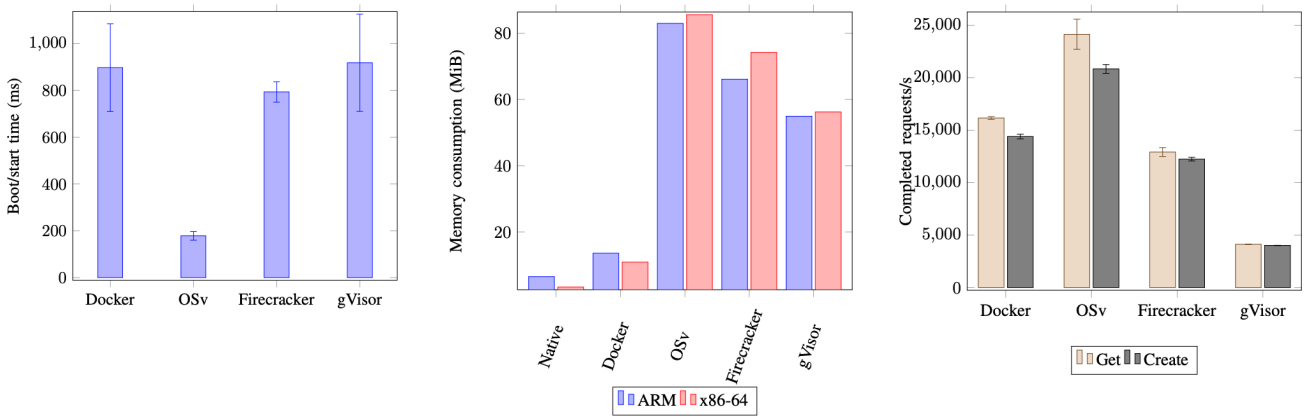
Performance of virtualization approaches

- Performance comparisons of hypervisor vs. lightweight virtualization
- Key findings:
  - Overhead introduced by containers is almost negligible
  - Fast instantiation time of containers
  - Small per-instance memory footprint
  - High density
  - But security tradeoffs: containers offer less isolation

Virtualization	Boot time	Image size	Memory footprint	Programming language dependance	Live migration
VM	~5/10 sec	~1 GB	~100 MB	No	Yes
Container	~0.8/1 sec	~50 MB	~5 MB	No	Non-native
Unikernel	<10 msec	<20 MB	~10 MB	Partially	No

Performance of virtualization approaches

- Comparing lightweight virtualization approaches
- Overall result: no clearly superior solution, each one has its own strengths and weaknesses



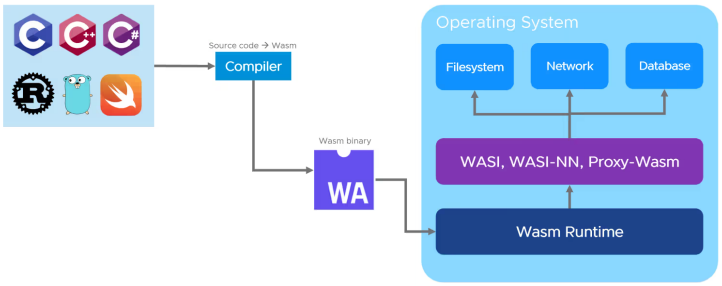
A functional and performance benchmark of lightweight virtualization platforms for edge computing, EDGE 2022 https://ieeexplore.ieee.org/document/9860335

WebAssembly (Wasm)

- Safe, portable, binary code format designed for efficient execution and compact representation https://webassembly.org
  - Safe: runs in a sandboxed environment, preventing untrusted code from harming the host system
  - Portable: the same Wasm binary can run on any platform that supports the Wasm runtime (browsers, servers, embedded)
  - Binary format: designed to be compact and fast to load and execute
- Other features
  - Open standard https://www.w3.org/TR/wasm-core-2
  - Portable compilation target for many programming languages
    - Originally built to safely execute JavaScript code in browsers
  - Memory-safe, sandboxed execution
  - Computational model based on stack VM

WebAssembly: features

- Wasm code is validated and executed in a memory-safe, sandboxed environment → strong isolation and protection from unsafe operations
  - Wasm interacts with the host system through WebAssembly System Interface (WASI), which provides a standardized set of capability-based APIs
  - A Wasm module cannot directly perform OS systems calls, instead imports host-provided WASI functions
- Development workflow: write code in one of many supported languages, compile it to Wasm, and run it inside a Wasm runtime



WebAssembly: features

- Wasm uses stack-based VM to execute instructions https://en.wikipedia.org/wiki/Stack_machine
  - Code is composed of sequences of instructions executed in order
  - The operand stack is used to store values for computation
  - Instructions:
    - Pop argument values from the stack and push back onto it
    - Each operation manipulates values on the stack
    - Example: `i64.add`
      - Takes two i64 values from the stack
      - Add them
      - Pushes the result back onto the stack
    - Control instructions alter control flow
- JVM is a famous example of stack-based VM

WebAssembly: example

- Factorial function written in C and its corresponding Wasm code after compilation
  - In `.wat` text format (human-readable textual representation of Wasm)

```c
int factorial(int n) {
  if (n == 0)
    return 1;
  else
    return n * factorial(n-1);
}
```

```
(func (param i64) (result i64)
  local.get 0             # put arg[0] on stack
  i64.eqz                 # compare top of stack to zero
  if (result i64)         # if it is zero
      i64.const 1         # put 1 on stack
  else
      local.get 0         # put arg[0] on stack
      local.get 0         # put arg[0] on stack
      i64.const 1         # put 1 on stack
      i64.sub             # subtract the top 2 values in stack
      call 0              # Call function #0 (return value is on the stack)
      i64.mul             # Multiply
  end)'
```


WebAssembly: example

- Factorial function written in C and its corresponding Wasm code after compilation
  - In `.wasm` binary format

```c
int factorial(int n) {
  if (n == 0)
    return 1;
  else
    return n * factorial(n-1);
}
```

```
00 61 73 6D 01 00 00 00
01 06 01 60 01 7E 01 7E
03 02 01 00
0A 17 01
15 00
20 00
50
04 7E
42 01
05
20 00
20 00
42 01
7D
10 00
7E
0B
0B
```

WebAssembly: pros and cons

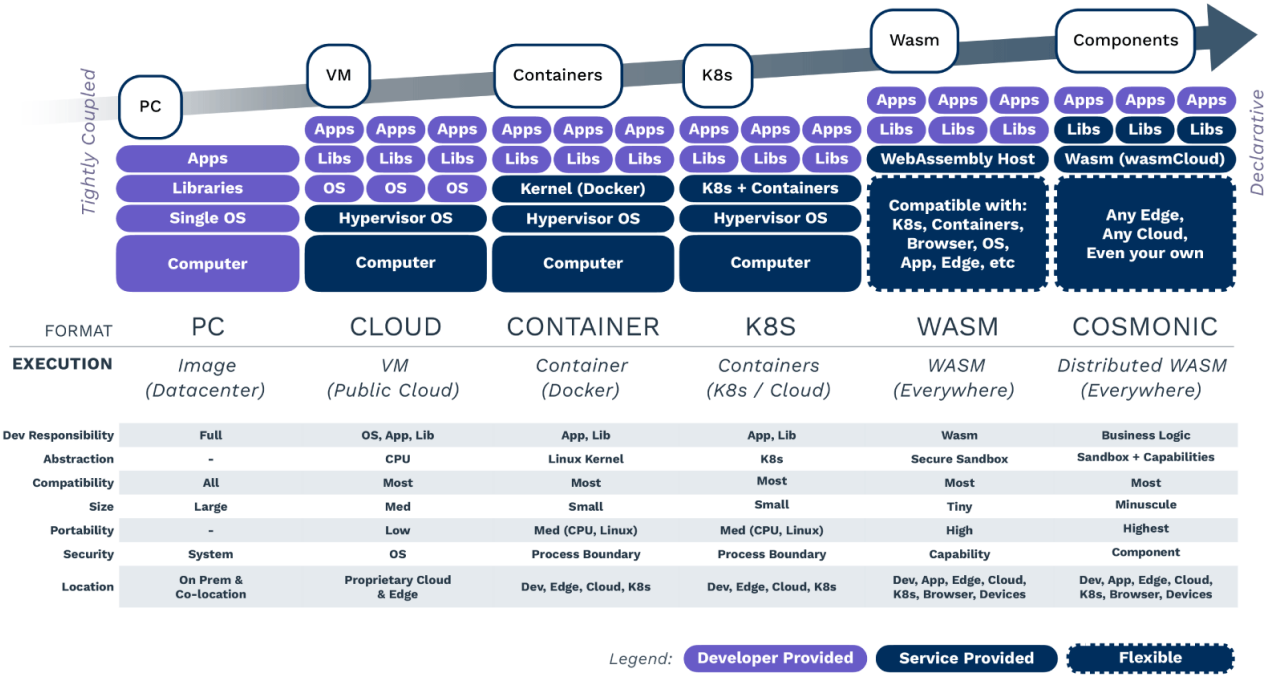
- ✓ Efficient: near-native execution speed
- ✓ Secure: memory-safe, sandboxed execution, which prevents data corruption and security breaches
- ✓ Language-, platform-, and hardware-independent
  - Does not favour any particular language
  - Can run as a standalone VM
  - Can be compiled for all modern architectures, including desktop, mobile devices, and embedded systems
- ✗ In development
- ✗ Support varies by language
- ✗ Multiple runtimes (e.g., Wasmtime, Wasmer, WasmEdge) with different features: choice is complex

WebAssembly

- How to try: Wasm applications with Linux containers in Docker (beta)
  https://docs.docker.com/desktop/features/wasm/
  - Enable it on Docker Desktop by checking *Enable Wasm* on the *Features in development* tab under *Settings* (requires containerd image store)

```
$ docker run \
  --runtime=io.containerd.wasmedge.v1 \
  --platform linux/arm64 \
  secondstate/rust-example-hello
```

The full scenario



FORMAT	PC	CLOUD	CONTAINER	K8S	WASM	COSMONIC
EXECUTION	*Image (Datacenter)*	*VM (Public Cloud)*	*Container (Docker)*	*Containers (K8s / Cloud)*	*WASM (Everywhere)*	*Distributed WASM (Everywhere)*
Dev Responsibility	Full	OS, App, Lib	App, Lib	App, Lib	Wasm	Business Logic
Abstraction	-	CPU	Linux Kernel	K8s	Secure Sandbox	Sandbox + Capabilities
Compatibility	All	Most	Most	Most	Most	Most
Size	Large	Med	Small	Small	Tiny	Minuscule
Portability	-	Low	Med (CPU, Linux)	Med (CPU, Linux)	High	Highest
Security	System	OS	Process Boundary	Process Boundary	Capability	Component
Location	On Prem & Co-location	Proprietary Cloud & Edge	Dev, Edge, Cloud, K8s	Dev, Edge, Cloud, K8s	Dev, App, Edge, Cloud, K8s, Browser, Devices	Dev, App, Edge, Cloud, K8s, Browser, Devices

Legend: Developer Provided | Service Provided | Flexible

References

- Sections 4.13 of Marinescu book

- Docker workshop https://docs.docker.com/get-started/workshop
- Docker Docs https://docs.docker.com
- Kane and Matthias, Docker up and running 3rd edition, O'Reilly, 2023

- Agache et al., Firecracker: Lightweight virtualization for serverless applications, NDSI 2020 https://www.usenix.org/conference/nsdi20/presentation/agache
- Kuenzer et al., Unikraft: fast, specialized unikernels the easy way, EuroSys 2021 https://dl.acm.org/doi/pdf/10.1145/3447786.3456248
- Menétrey et al., WebAssembly as a common layer for the cloud-edge continuum, FRAME 2022 https://dl.acm.org/doi/10.1145/3526059.3533618